

- 1º** Abrir o **VSCODE**
- 2º** No terminal digitar: **cd desktop**
- 3º** No terminal dentro de desktop: **mkdir projeto**
- 4º** No terminal: **cd projeto**
- 5º** dentro da pasta projeto: **code .**
- 6º** No terminal da pasta projeto: **touch index.html**
- 7º** No terminal: **mkdir src**
- 8º** No terminal : **cd src**
- 9º** No terminal : **mkdir css**
- 10º** No terminal : **mkdir js**
- 11º** No terminal : **cd css**
- 12º** No terminal : **touch estilo.css**
- 13º** No terminal : **cd ..**
- 14º** No terminal : **cd js**
- 15º** No terminal : **touch script.js**
- 16º** No terminal : **cd ..**
- 17º** No terminal : **cd .. até chegar na raiz do projeto**
- 18º** No terminal : **git init**

Obs: Caso utilize o seu notebook a configuração do nome e email somente uma vez

- 19º** Configurar o nome: **git config --global user.name "Seu Primeiro Nome"**
- 20º** Configurar o email: **git config --global user.email email@email.com**
- 21º** No terminal: **git status**
- 22º** Se estiver vermelho: **git add .**
- 23º** Se estiver verde: **git commit -m "Mensagem do que você fez"**

24º Entrar no Gitub em repositórios, criar um repositório (NEW), colocar o nome e deixar público, clicar no botão criar repositório (**DICA DE OURO- NUNCA TRADUZA A PÁGINA DO GITHUB**)

25º No bloco que estiver as 3 linhas:

1º linha- Conecta seu gitbash com o seu github

2º linha- muda o nome master para main

3º linha- pega tudo que estiver no projeto e sobe para o github

26º Caso você esteja em outro pc: no seu repositório no **botão verde(code)**, copie o link

27º Abra o seu VSCODE no terminal cd desktop : **git clone (copie o link)**, isso vai clonar seu projeto e pode continuar de onde parou.

28º A cada modificação do seu código faça todo o processo do

git status

git add .

git commit -m " mensagem"

e para subir as alterações só digitar : git push

OBS: NUNCA DIGITE git init EM UM PROJETO CLONADO, GIT INIT SOMENTE UMA VEZ EM UM PROJETO NOVO

COMMITTS SEMÂNTICOS

O que é?

Commits Semânticos são mensagens de commit padronizadas que descrevem de forma clara e concisa a natureza e o propósito das alterações feitas em um projeto de software. Eles seguem uma convenção específica, como a Convencional Commits, que torna o histórico de commits legível tanto para humanos quanto para máquinas.

A ideia é substituir mensagens vagas (como "atualização" ou "correção de bug") por descrições detalhadas e estruturadas, que indicam o tipo de mudança, o escopo (opcional) e uma descrição sucinta.

Exemplos

feat	Nova funcionalidade	feat: adiciona sistema de busca
fix	Correção de erro	fix: corrige alinhamento do logo
style	Mudança visual (CSS)	style: altera paleta para tema escuro
docs	Documentação	docs: atualiza instruções do README
refactor	Refatoração	refactor: melhorias de código
chore	Criação e configuração	chore: Criando e configurando